

Homework #2

Assigned: 21 September 2026

Due: 5 October 2026

Please submit a .zip folder containing *a)* Your (directly runnable) code with any commands we should run to run it, and *b)* A writeup with answers to the problems below.

Notes:

- ▶ Exercise 1.2 is only required for those taking 18-660. Students taking 18-460 may attempt it for extra credit.
- ▶ Exercise 3.3 is optional for everyone, and may be attempted for extra credit.
- ▶ To help you in reviewing relevant material, we have put the lecture most relevant to each problem in parentheses at the end of the exercise title.
- ▶ AI disclosure: ChatGPT-6.o was used in helping to create these exercises. (All problems were intellectually ideated and designed solely by the course staff. AI was used only to help write code.)

1 Theory

Exercise 1.1 10 pts: Easy Does It (Lecture 7)

Consider a continuation method that steps parameters by at least $h > 0$ with each continuation stage. At each stage, the continuation method performs a fixed positive number of solver steps while maintaining an error of at most r .

You can assume two “sufficient tracking conditions”:

$$r + Vh \leq R, \quad q^m(r + Vh) \leq r,$$

where $0 < q < 1$, $V > 0$, and $R > 0$ are fixed constants. (The second condition can be derived from the one we proved in class.)

Each stage has setup cost $s \geq 0$, and each correction costs $c > 0$. We want to choose $h > 0$, $0 < r < R$, and an integer $m \geq 1$ to minimize the **cost per unit of parameter change**:

$$J(h, r, m) = \frac{s + cm}{h}.$$

- (a) For a fixed integer $m \geq 1$, find the optimal h and r . Express the resulting cost as a function of m alone.
- (b) Relax m to a real variable satisfying $m \geq 1$. Derive an equation characterizing the continuous minimizer, accounting for the possibility that the minimum occurs at $m = 1$. Show that the optimal integer m can be found by comparing the floor and ceiling of the continuous minimizer.

Exercise 1.2 25 pts: The Implicit Truth (Lecture 8)

NOTE: This problem is required for 18-660 ONLY! Those taking 18-460 may attempt it for extra credit.

Let $h : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^n$ be a continuously differentiable function, and let $\hat{z}$ approximate a solution of $h(z, 0) = 0$.

$B \in \mathbb{R}^{n \times n}$. $\eta, \alpha, L \geq 0$ are constant scalars. We define $J(z) = D_z h(z, 0)$ is the Jacobian of h .

Assume the function $g(z) = BJ(z)$ is L -Lipschitz on the ball $K_r = \{z : \|z - \hat{z}\| \leq r\}$, and

$$\|Bh(\hat{z}, 0)\| \leq \eta, \quad \|I - g(\hat{z})\| \leq \alpha.$$

(For simplicity, you can also assume the norm to be the L_2 norm.)

A map from a nonempty closed ball into itself that is k -Lipschitz for some $0 \leq k < 1$ has a unique fixed point—that is, a unique point z^* satisfying $T(z^*) = z^*$. This makes T a *contraction*.

The question we would like to examine here concerns the applicability of the implicit function theorem in real-world cases. T , applied many times into itself, eventually resolves to the true fixed point at which we would apply the implicit function theorem as discussed in class. But if we stop early, at some $\hat{z}$, which numerically isn't *quite* truly 0 can we still apply the implicit function theorem? Under which conditions?

- (a) Let $T(z) = z - Bh(z, 0)$. Determine a bound on the Lipschitz constant of T in K_r in terms of α, L, r such that T is a contraction.

HINT: Differentiate T to get started, and consider the result from homework 1, problem 2.

- (b) What remaining inequality is needed to ensure that $T(K_r) \subseteq K_r$?

HINT: Consider the magnitude of difference from $T(z) - \hat{z}$, and consider the triangle inequality.

- (c) Assuming the conditions in the previous two parts are true, prove that $J(z)$ is nonsingular throughout K_r and that $h(z, 0) = 0$ has a unique solution $z^* \in K_r$. Use this to verify the three preconditions of the implicit function theorem (Theorem A.2 in Nocedal & Wright) at $(z^*, 0)$.

HINT: Begin by establishing that if $\|I - A\| < 1$, then A is nonsingular.

2 Implementation

Exercise 2.1 20 Pts: Linkage's Awakening (Lecture 8)

Linkages are used in engineered systems to transduce one rotational input motion into another motion profile. In this problem, you will design these linkages computationally using optimization. We've given you some capabilities: a forward "simulator," animation utilities, and a discrete Fréchet distance objective that measures the distance from a linkage's end effector to a prescribed curve. You will augment these pieces with judicious application of the implicit function theorem to optimize the linkage parameters.

Details can be found in the attached README.md on how to use `optimize.py` to *optimize* the design. Let $\ell \in \mathbb{R}^4$ denote the link lengths. At each prescribed driver angle, the simulator computes a state $x_t^*(\ell) \in \mathbb{R}^6$ satisfying the stationarity condition

$$F_t(x_t^*(\ell), \ell) = 0, \quad F_t(x, \ell) = \nabla_x E_t(x, \ell),$$

Here, E_t is an energy function that you will implement that ensures the linkage is fully connected and closed given the angle of the crankshaft drive as forced/fixed.

Complete the four TODO blocks in the codebase:

1. **closure_energy.** Implement an energy function that is minimized when the endpoints of the joint links are incident to one another.
2. **batched_state_sensitivities.** Differentiate the stationarity condition and implement $\partial x_t^* / \partial \ell$. Use the provided partial-derivative helpers and return an array of shape $(T, 6, 4)$. Do not compute a matrix inverse directly; rather, solve systems of equations.
3. **design_objective_and_gradient.** Combine the computed sensitivities with the supplied objective partials to compute the total derivative of the objective with respect to the lengths. Your computation should include both the dependence through the simulated states and the direct dependence on the lengths.
4. **gradient_descent_update.** Implement a gradient step on the link lengths, making sure to project the system to the feasible variable bounds.

With all these pieces combined, the optimizer should update your design to sweep a curve that is close to the target.

Run `python checks.py --part all`, then `python optimize.py --show` with the default settings. Submit a .zip containing your directly runnable code, the commands needed to reproduce your results, and a brief writeup including images and animations and parameter values of the system before and after optimization. Provide a plot of objective value versus outer iteration during the optimization as well.

3

Application

Each homework will feature a theme from a different application area. This week's themes are **games and computational design** (as hopefully already seen in part from the prior section).

In this homework you will use JAXopt, which already comes loaded with linear programming and quadratic programming methods to help you with problems 3.1 and 3.2 respectively. We've given you code for each problem to help you get started.

Exercise 3.1 20 Pts: Teeth Nashing (Lecture 9)

In this assignment, you will implement a method to play “optimally” at a simplified version of Texas Hold 'Em poker, a partial information game. By optimal, we mean that your opponent improve their expected chip payoff by changing their strategy. This is a one hand game and your goal is to maximize the expected value of the chips you win. It is “zero-sum”: the chips you lose are equal to the chips your opponent gains, and vice versa. Your win is your opponent's loss; your opponent's loss is your win.

The game

We'll play this game with two different decks. To get you started, the first deck has only six cards: 2, 3, 4 each in two suits; later, you'll play with a full 52 card deck. Each player antes one chip and gets one “hole” card at random without replacement face down, and there is one community card. Each player's hand is their secret hole card plus the community card.

Since each player only has two cards, the hands are limited, and they are ranked as follows:

- ▶ The best hand is a pair of the same number.
- ▶ The second-best hand is a flush — both cards have the same suit. (Note this is usually a *very* highly ranked hand in regular poker but not so hard to do when each hand only has two cards, so we rank it lower.)
- ▶ The worst hand is a “high card.”
- ▶ Ties for high card and pairs are broken by the value of the number on the high card or the pair. If both players have the same high card, the value of the other card serves tiebreaker. Suits do not break ties.
- ▶ 2's are low, after 10's, the remaining cards are ranked Jack, Queen, King, with Ace being the best.

With cards dealt, each player then places a secret bet, simultaneously. To start, the bids may be 0 chips or 1 chip; we also consider a variant where the number of chips may be 0, 1, or 2.

If both players bid the same number of chips, the game moves to a showdown: both players reveal their hand, and the better hand takes all the chips in the pot.

If players bid different numbers of chips, the player who bid more takes all the chips and there is no showdown.

We have provided a poker simulator, so you do not need to implement the game.

The optimization problem

Let $\sigma(c, h_1, h_2) \in \{-1, 0, 1\}$ be the showdown outcome for player 1. The net transfer to player 1 is

$$U(c, h_1, h_2, a, b) = \begin{cases} (1 + a) \sigma(c, h_1, h_2), & a = b, \\ (1 + \min(a, b)) \text{sign}(a - b), & a \neq b. \end{cases}$$

Let c be the community card. There are $H = D - 1$ possible hole cards, where D is the deck size, and $B = |\mathcal{B}|$ actions. Write

$$p_{ia} = \Pr(a \mid c, h_i), \quad q_{jb} = \Pr(b \mid c, h_j).$$

For each private card there is a different action strategy with its own associated probability distribution. The provided matrix $K_c \in \mathbb{R}^{HB \times HB}$ has entries

$$(K_c)_{(i,a),(j,b)} = \frac{\mathbf{1}\{i \neq j\}}{H(H-1)} U(c, h_i, h_j, a, b).$$

It already contains the joint chance probabilities for the private deal. With p, q flattened in type-action order, the expected payoff on that board is $p^\top K_c q$. Player 1 wants

$$\max_{p \in \mathcal{P}} \min_{q \in \mathcal{Q}} p^\top K_c q, \quad \mathcal{P} = \left\{ p : p_{ia} \geq 0, \sum_a p_{ia} = 1 \text{ for every } i \right\}.$$

You must not let a player's policy depend on the other player's private card.

1. Write player 1's problem as a linear program, s.t. optimizing that problem provides player 1's reward-maximizing strategy. Use one auxiliary value variable per opponent hole card, not per strategy.
2. Implement the `row_lp` function. Return the supplied QP container.
3. Solve for both sets of allowed bids on the six-card deck, and then the fifty-two card deck. Fix a value of the community card (your choice) and report the action probabilities for each hole card in each scenario. Report the best-response bounds

$$L(p) = \min_{q \in \mathcal{Q}} p^\top K_c q, \quad R(q) = \max_{p \in \mathcal{P}} p^\top K_c q,$$

using the supplied evaluator, and the gap $R(q) - L(p)$ on every board. Where, if anywhere, does the optimal policy bluff?

Note that some problems may have more than one equally valid strategy. Along with answers to the above problems, please provide your code.

Exercise 3.2 25 Pts: Thanks For The Complement (Lecture 10)

In this scenario, we're going to use quadratic programming to simulate the behavior of a ball bouncing around in a room (well, of radius 0; it will be a point mass). This is a common recipe for robotics simulators and video game physics engines.

(Since this is a point mass, we can ignore rotations and inertia. Phew!)

We need to simulate Newtonian mechanics and collision detection and resolution; in particular, we need to handle normal forces, restitution, and friction. To make the problem simpler, we'll assume the environment is *convex*.

The world and the time step

Assume a room that is bounded by planes on all sides, making it the intersection of n half-spaces,

$$\mathcal{W} = \{x \in \mathbb{R}^3 : n_i^\top x \geq b_i, i = 1, \dots, n\}, \quad \|n_i\|_2 = 1.$$

The normals of the planes (walls) point inward. The point has position x , velocity v , and mass $m > 0$. Its gap to plane i is $\phi_i(x) = n_i^\top x - b_i$.

We simulate the world one time step at a time. When not in collision, the mass moves according to an initial velocity and acceleration due to gravity. The update is simply $v \leftarrow v + hg$; easy! It gets trickier when contact is involved.

The contact model

At a collision, let v^- be the incoming velocity and let $\mathcal{A}$ be the set of touching planes. The rows of $T_i \in \mathbb{R}^{2 \times 3}$ are two supplied orthonormal tangents to plane i . The routine `contact_parameters` supplies a normal target $d_i \geq 0$ and a fixed friction-impulse budget $\beta_i \geq 0$. It uses

$$d_i = e_i^{\text{eff}} \max(-n_i^\top v^-, 0), \quad \beta_i = \mu_i m (1 + e_i^{\text{eff}}) \max(-n_i^\top v^-, 0).$$

Here μ_i is a friction coefficient and e_i^{eff} is the coefficient of restitution.

The outgoing velocity of a collision solves the following optimization problem

$$\begin{array}{ll} \min_{v \in \mathbb{R}^3} & J_{\text{contact}}(v) = \underbrace{\frac{m}{2} \|v - v^-\|_2^2}_{\text{cost of changing the incoming velocity}} + \underbrace{\sum_{i \in \mathcal{A}} \beta_i \|T_i v\|_\infty}_{\text{penalty for tangential slip}} \\ \text{subject to} & n_i^\top v \geq d_i \quad (i \in \mathcal{A}). \end{array}$$

More details can be found in the `README.md`.

1. Turn the contact model into a QP. Introduce auxiliary variables to write the problem in a way that does not require infinity norms. Complete `contact_qp` in `physics.py`.
2. Complete `time_to_collision` in the same file. Derive an equation for the time it will take for the mass to hit each candidate plane and select the first event within the remaining interval. Be sure to handle special cases, including parallel or separating motion and existing contact.

3. Run the supplied analytic checks and random worlds. Report minimum gap, contact residual, impact kinetic-energy change, solver status, and QP-call count. There should be one *logical* QP solve per collision event, not one per touching plane.
4. Compare several time steps and QP tolerances, and explain the difference between integration error and an incomplete optimization. Which dimensions of the solver can be parallelized?

In addition to the above deliverables, please submit your runnable code.

Exercise 3.3 15 Pts: Breaking The Mold (Lecture 9)**NOTE: This problem is optional for EVERYBODY!**

For this problem, you may implement the surface-segmentation model from Section 4.1 of the *Metamolds* paper cited below. The algorithm formulates an integer linear program. For full credit, you must test it on a 3D mesh and visualize the result. You are welcome to use any off-the-shelf integer linear programming solver you find to aid you in solving this problem.

T. Alderighi et al. *Metamolds: Computational Design of Silicone Molds*. ACM Transactions on Graphics 37(4), Article 136, 2018. Read §4.1 and skim §§4.2–4.5 for context. [doi:10.1145/3197517.3201381](https://doi.org/10.1145/3197517.3201381).